

SPORTS CLUB MEMBERSHIP AND TOURNAMENT MANAGEMENT SYSTEM



Name: M. Reethu Chandra

Roll No: 25WU0102146

Course: Database Management System(DBMS)

Faculty Name: Kiran Mayee

Academic Year: 2026-27

TABLE OF CONTENTS

1. Cover Page
2. Abstract
3. Introduction and Problem Statement
4. Objectives and Scope
5. Software and Hardware Requirements
6. ER Diagram
7. Relational Schema and Normalization
8. Data Dictionary
9. SQL Commands Used (DDL, DML) with Sample Outputs
10. Queries with Outputs
11. UI Design and Screenshots
12. Implementation Details
13. Testing
14. Conclusion and Future Enhancements
15. References
16. Appendix: GitHub Repository Link

2. Abstract

Sports clubs handle membership plans, members, payments, sports, coaches, teams, training sessions, facilities, tournaments, fixtures, results and equipment. When these records are maintained in paper registers or separate spreadsheets, data may be duplicated and important information becomes difficult to retrieve. A centralized database helps the club organize these records and connect related activities.

This project presents the design and implementation of a relational Database Management System for a Sports Club Membership and Tournament Management System. The conceptual design is represented using an Entity–Relationship diagram and mapped to a final schema of fifteen tables: MEMBERSHIP_PLAN, MEMBER, PAYMENT, SPORT, MEMBER_SPORT, COACH, TEAM, FACILITY, TRAINING_SESSION, TOURNAMENT, PARTICIPANT, FIXTURE, RESULT, EQUIPMENT and EQUIPMENT_ISSUE. Primary keys uniquely identify records, and foreign keys link dependent records to their parent tables.

The supplied MySQL script creates the sports_club database and inserts ten records into each table, giving a total of 150 sample records. SQL queries demonstrate filtering, joins, aggregation and grouping. A Python Tkinter desktop interface connects to MySQL and supports viewing, inserting and deleting records. The supplied presentation documents the connection screen, member records, record insertion and payment deletion.

The project provides a structured foundation for managing club information. Additional validation, membership expiry tracking, schedule conflict checks and equipment stock automation can extend the system in future versions.

Keywords: DBMS, MySQL, Python, Tkinter, ER Diagram, Normalization, Sports Club, Membership, Tournament, SQL

3. Introduction and Problem Statement

3.1 Introduction

A sports club offers different sports, membership plans, coaching, training facilities and tournaments. Each activity produces related information: a member selects a membership plan, makes payments, registers for sports and participates in tournaments. Teams are associated with sports and coaches, while training sessions require facilities and scheduled times. Equipment issues and match results also need to be recorded.

A relational database stores these details in separate tables connected through keys. Instead of repeating a member's details in every payment or equipment record, the system stores a Member_ID reference. This makes retrieval easier and reduces unnecessary duplication.

3.2 Problem Statement

The first presentation identifies the following difficulties in manual sports club management:

- Large amounts of member, team, tournament and facility data are difficult to organize.
- Manual record management can cause duplication, entry errors and slow retrieval.
- Scheduling teams, training sessions and facilities can lead to time conflicts.
- Membership payments, tournament results and equipment issues become harder to track as the club grows.
- Separate records do not provide a reliable link between members, plans, payments and participation.

The problem addressed by this project is to design a centralized relational database that organizes these records, maintains valid relationships and supports meaningful queries. The implemented schema records schedules; automatic detection of overlapping bookings is a future enhancement.

4. Objectives and Scope

4.1 Objectives

- To identify the entities and relationships required for sports club management.
- To design an ER diagram and map it to a relational schema.
- To organize the tables according to Third Normal Form under the stated functional dependencies.
- To implement the database using MySQL DDL and populate it using DML statements.
- To manage members, membership plans, sports, coaches, teams and training sessions.
- To record tournaments, participants, fixtures, results, payments and equipment issues.
- To provide a Python Tkinter interface for viewing, inserting and deleting records.
- To use SQL queries to retrieve and summarize club information.

4.2 Scope

- Membership: member details, plans, join dates, status and payments.
- Team and training: sports, coaches, teams, facilities and training schedules.
- Tournament: tournament details, registered participants, fixtures and results.
- Equipment: equipment inventory quantities and issue/return records.
- Reports: member counts, payment summaries and linked training or tournament details.

Out of scope: online payment gateway integration, automatic fee-due calculation, automatic membership expiry, user authentication, facility conflict prevention and automatic equipment stock updates. The schema stores participant roles but does not implement role-based access control.

5. Software and Hardware Requirements

5.1 Software Requirements

Component	Specification
Operating System	macOS for the supplied desktop demonstration
Database	MySQL Server
Database Client	MySQL Workbench / MySQL command-line client
Programming Language	Python 3.x
GUI Library	Tkinter
Database Connector	mysql-connector-python
Code Editor	Any Python-compatible editor
Design / Documentation	ER diagram and project presentations
Version Control	Git and GitHub for project submission

5.2 Hardware Requirements

The following are practical minimum recommendations for running the project; they are not measured performance requirements.

Component	Recommended Minimum
Processor	Dual-core processor or equivalent
RAM	4 GB; 8 GB recommended for MySQL and IDE use
Storage	At least 1 GB free for database and project files
Display	1366 × 768 resolution or higher
Other	Keyboard, mouse/trackpad; internet for GitHub access

6. ER Diagram

The diagram below is taken from Presentation 2. Rectangles represent entities, ellipses represent attributes, and diamonds represent relationships with cardinalities. It shows the main membership, sports, training, tournament and equipment workflows.

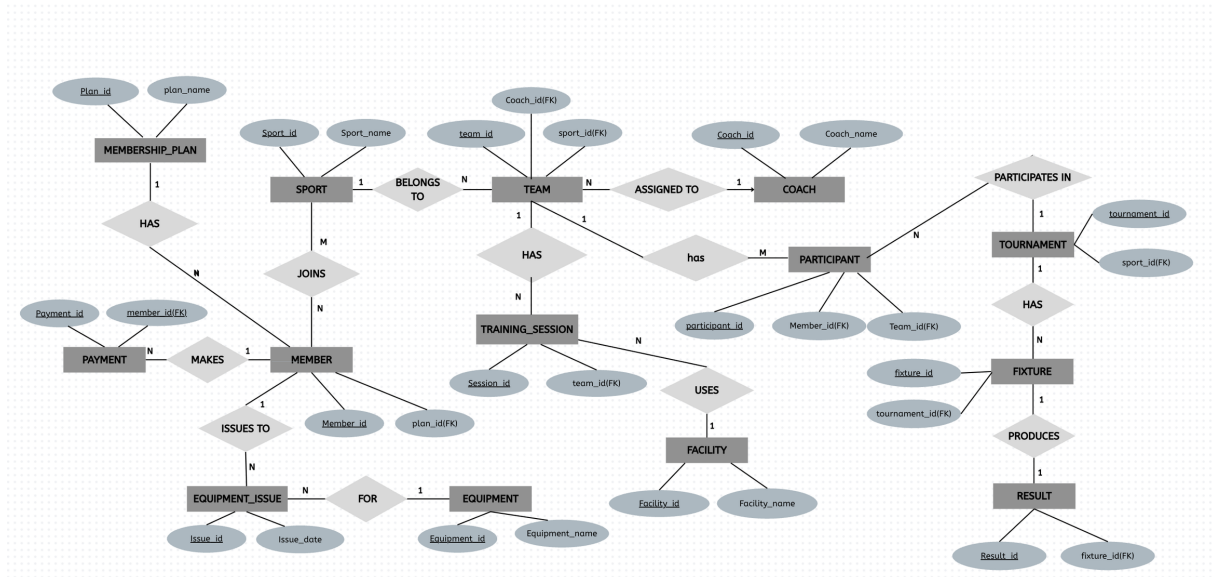


Figure 6.1: Original ER diagram of the Sports Club Management System

6.1 Mapping to the Final Database

The final SQL resolves the many-to-many association between MEMBER and SPORT through MEMBER_SPORT. This junction table contains Member_ID and Sport_ID as a composite primary key, with each column also referencing its parent table.

The final SQL is used as the authority for table names, attributes and constraints in this report. Some conceptual relationships in the initial diagram are not directly enforced by the supplied DDL. For example, team membership outside tournaments has no separate junction table, and RESULT.Fixture_ID is not UNIQUE.

The initial ER diagram is preserved as supplied. The implemented relationships and their limitations are listed on the next page.

6.2 Relationships and Cardinality

Parent Entity	Relationship	Child Entity / FK
MEMBERSHIP_PLAN	HAS	MEMBER / Plan_ID
MEMBER	MAKES	PAYMENT / Member_ID
MEMBER	REGISTERS	MEMBER_SPORT / Member_ID
SPORT	LINKS TO	MEMBER_SPORT / Sport_ID
SPORT	HAS	TEAM / Sport_ID
COACH	COACHES	TEAM / Coach_ID
TEAM	HAS	TRAINING_SESSION / Team_ID
FACILITY	HOSTS	TRAINING_SESSION / Facility_ID
SPORT	HAS	TOURNAMENT / Sport_ID
TOURNAMENT	REGISTERS	PARTICIPANT / Tournament_ID
MEMBER	IS LISTED IN	PARTICIPANT / Member_ID
TEAM	IS LISTED IN	PARTICIPANT / Team_ID
TOURNAMENT	HAS	FIXTURE / Tournament_ID
FIXTURE	PRODUCES	RESULT / Fixture_ID
MEMBER	RECEIVES	EQUIPMENT_ISSUE / Member_ID
EQUIPMENT	IS ISSUED IN	EQUIPMENT_ISSUE / Equipment_ID

Every listed relationship supports one parent to many child records in the supplied schema. Since foreign-key columns are nullable, a child may also have no parent reference. The composite key of MEMBER_SPORT prevents the same member–sport pair from appearing twice.

Conceptual versus implemented cardinality: one result per fixture may be intended, but the SQL allows multiple RESULT rows for the same Fixture_ID. A UNIQUE constraint would be required to enforce a one-to-one mapping. Similarly, PARTICIPANT does not prevent a member from registering repeatedly in the same tournament under different Participant_ID values.

7. Relational Schema and Normalization

7.1 Relational Schema

PK = Primary Key; FK = Foreign Key. MEMBER_SPORT has a composite primary key. The other fourteen tables use single-column primary keys.

Relation	Attributes
MEMBERSHIP_PLAN	Plan_ID (PK), Plan_Name, Duration, Fee_Amount, Description
MEMBER	Member_ID (PK), Name, DOB, Gender, Phone, Email, Address, Join_Date, Status, Plan_ID (FK)
PAYMENT	Payment_ID (PK), Payment_Date, Amount, Remarks, Mode, Member_ID (FK)
SPORT	Sport_ID (PK), Sport_Name, Description
MEMBER_SPORT	Member_ID (PK, FK), Sport_ID (PK, FK)
COACH	Coach_ID (PK), Coach_Name, Phone, Email, Experience
TEAM	Team_ID (PK), Team_Name, Sport_ID (FK), Coach_ID (FK)
FACILITY	Facility_ID (PK), Facility_Name, Location, Type, Availability
TRAINING_SESSION	Session_ID (PK), Session_Date, Start_Time, End_Time, Team_ID (FK), Facility_ID (FK)
TOURNAMENT	Tournament_ID (PK), Tournament_Name, Sport_ID (FK), Start_Date, End_Date, Description
PARTICIPANT	Participant_ID (PK), Tournament_ID (FK), Member_ID (FK), Team_ID (FK), Role
FIXTURE	Fixture_ID (PK), Fixture_Date, Time, Venue, Tournament_ID (FK)
RESULT	Result_ID (PK), Home_Score, Away_Score, Winner, Remarks, Fixture_ID (FK)
EQUIPMENT	Equipment_ID (PK), Equipment_Name, Category, Quantity
EQUIPMENT_ISSUE	Issue_ID (PK), Issue_Date, Return_Date, Condition, Member_ID (FK), Equipment_ID (FK)

7.2 Changes from the Initial Schema

- MEMBER_SPORT was added to implement member registration in multiple sports. The initial relational schema listed fourteen tables; the final SQL contains fifteen.
- The final EQUIPMENT_ISSUE column `Condition` is quoted with backticks in MySQL, as shown in the supplied script.

7.3 Normalization

First Normal Form (1NF). The tables store one value in each field and use primary keys to identify records. Multiple sports for a member are stored as separate MEMBER_SPORT rows rather than as a list inside MEMBER. Payments and equipment issues are also stored as separate records.

Second Normal Form (2NF). Fourteen tables have single-column primary keys, so their non-key attributes cannot depend on only part of those keys. MEMBER_SPORT has the composite key (Member_ID, Sport_ID) and no non-key attributes; therefore it has no partial dependency.

Third Normal Form (3NF). Under the intended dependencies, the key of each table determines its non-key attributes, without a non-key attribute determining another non-key attribute in the same relation. Plan details are kept in MEMBERSHIP_PLAN, member details in MEMBER, coach details in COACH and equipment details in EQUIPMENT. Child tables store their identifiers instead of duplicating descriptive details.

7.4 Representative Functional Dependencies

Relation	Functional Dependency
MEMBERSHIP_PLAN	Plan_ID → Plan_Name, Duration, Fee_Amount, Description
MEMBER	Member_ID → Name, DOB, Gender, Phone, Email, Address, Join_Date, Status, Plan_ID
PAYMENT	Payment_ID → Payment_Date, Amount, Remarks, Mode, Member_ID
TEAM	Team_ID → Team_Name, Sport_ID, Coach_ID
TRAINING_SESSION	Session_ID → Session_Date, Start_Time, End_Time, Team_ID, Facility_ID
EQUIPMENT	Equipment_ID → Equipment_Name, Category, Quantity

These dependencies describe the intended design. Normalization depends on business rules, not only the sample rows. Phone, Email and entity names are not declared UNIQUE, so this report does not assume they are candidate keys or claim that every table is in BCNF.

Normalization and constraints: a table may be designed in 3NF while still allowing invalid values. The supplied DDL does not include CHECK, DEFAULT or additional NOT NULL/UNIQUE constraints. Negative fees, invalid status values and schedule conflicts require additional validation.

8. Data Dictionary

Data types and constraints below match the supplied SQL. PK = Primary Key; FK = Foreign Key. Columns outside primary keys allow NULL unless otherwise specified.

8.1 MEMBERSHIP_PLAN

Stores membership plans, duration and fees.

Column	Data Type	Constraint	Description
Plan_ID	INT	PRIMARY KEY	Membership plan identifier
Plan_Name	VARCHAR(50)	Nullable	Name of the membership plan
Duration	INT	Nullable	Plan duration; sample values represent months
Fee_Amount	DECIMAL(10,2)	Nullable	Membership plan fee
Description	VARCHAR(100)	Nullable	Descriptive information

8.2 MEMBER

Stores personal details and the selected membership plan.

Column	Data Type	Constraint	Description
Member_ID	INT	PRIMARY KEY	Member identifier
Name	VARCHAR(50)	Nullable	Member name
DOB	DATE	Nullable	Date of birth
Gender	VARCHAR(10)	Nullable	Gender label
Phone	VARCHAR(15)	Nullable	Contact phone number
Email	VARCHAR(50)	Nullable	E-mail address
Address	VARCHAR(100)	Nullable	Member address or locality
Join_Date	DATE	Nullable	Membership joining date
Status	VARCHAR(20)	Nullable	Membership status label
Plan_ID	INT	Nullable, FK → MEMBERSHIP_PLAN	Membership plan identifier

8.3 PAYMENT

Stores payment transactions made by members.

Column	Data Type	Constraint	Description
Payment_ID	INT	PRIMARY KEY	Payment transaction identifier
Payment_Date	DATE	Nullable	Date of payment
Amount	DECIMAL(10,2)	Nullable	Payment amount
Remarks	VARCHAR(100)	Nullable	Additional remarks
Mode	VARCHAR(20)	Nullable	Payment method label
Member_ID	INT	Nullable, FK → MEMBER	Member identifier

8. Data Dictionary (continued)

8.4 SPORT

Stores the sports offered by the club.

Column	Data Type	Constraint	Description
Sport_ID	INT	PRIMARY KEY	Sport identifier
Sport_Name	VARCHAR(50)	Nullable	Sport name
Description	VARCHAR(100)	Nullable	Descriptive information

8.5 MEMBER_SPORT

Resolves the many-to-many member–sport relationship.

Column	Data Type	Constraint	Description
Member_ID	INT	PRIMARY KEY (composite), FK → MEMBER	Member identifier
Sport_ID	INT	PRIMARY KEY (composite), FK → SPORT	Sport identifier

8.6 COACH

Stores coach contact details and experience.

Column	Data Type	Constraint	Description
Coach_ID	INT	PRIMARY KEY	Coach identifier
Coach_Name	VARCHAR(50)	Nullable	Coach name
Phone	VARCHAR(15)	Nullable	Contact phone number
Email	VARCHAR(50)	Nullable	E-mail address
Experience	INT	Nullable	Coaching experience; interpreted as years

8. Data Dictionary (continued)

8.7 TEAM

Stores teams and their sport and coach references.

Column	Data Type	Constraint	Description
Team_ID	INT	PRIMARY KEY	Team identifier
Team_Name	VARCHAR(50)	Nullable	Team name
Sport_ID	INT	Nullable, FK → SPORT	Sport identifier
Coach_ID	INT	Nullable, FK → COACH	Coach identifier

8.8 FACILITY

Stores grounds, courts, halls, pools and availability labels.

Column	Data Type	Constraint	Description
Facility_ID	INT	PRIMARY KEY	Facility identifier
Facility_Name	VARCHAR(50)	Nullable	Facility name
Location	VARCHAR(50)	Nullable	Facility location
Type	VARCHAR(30)	Nullable	Facility type
Availability	VARCHAR(20)	Nullable	Availability label

8.9 TRAINING_SESSION

Stores training dates, times, teams and facilities.

Column	Data Type	Constraint	Description
Session_ID	INT	PRIMARY KEY	Training session identifier
Session_Date	DATE	Nullable	Training date
Start_Time	TIME	Nullable	Scheduled start time
End_Time	TIME	Nullable	Scheduled end time
Team_ID	INT	Nullable, FK → TEAM	Team identifier
Facility_ID	INT	Nullable, FK → FACILITY	Facility identifier

8. Data Dictionary (continued)

8.10 TOURNAMENT

Stores tournaments and their associated sports.

Column	Data Type	Constraint	Description
Tournament_ID	INT	PRIMARY KEY	Tournament identifier
Tournament_Name	VARCHAR(50)	Nullable	Tournament name
Sport_ID	INT	Nullable, FK → SPORT	Sport identifier
Start_Date	DATE	Nullable	Tournament start date
End_Date	DATE	Nullable	Tournament end date
Description	VARCHAR(100)	Nullable	Descriptive information

8.11 PARTICIPANT

Links tournament registrations to members and teams.

Column	Data Type	Constraint	Description
Participant_ID	INT	PRIMARY KEY	Registration identifier
Tournament_ID	INT	Nullable, FK → TOURNAMENT	Tournament identifier
Member_ID	INT	Nullable, FK → MEMBER	Member identifier
Team_ID	INT	Nullable, FK → TEAM	Team identifier
Role	VARCHAR(30)	Nullable	Participant role

8.12 FIXTURE

Stores scheduled events for tournaments.

Column	Data Type	Constraint	Description
Fixture_ID	INT	PRIMARY KEY	Fixture identifier
Fixture_Date	DATE	Nullable	Fixture date
Time	TIME	Nullable	Fixture time
Venue	VARCHAR(50)	Nullable	Venue text
Tournament_ID	INT	Nullable, FK → TOURNAMENT	Tournament identifier

8. Data Dictionary (continued)

8.13 RESULT

Stores scores, winner text and remarks for fixtures.

Column	Data Type	Constraint	Description
Result_ID	INT	PRIMARY KEY	Result identifier
Home_Score	INT	Nullable	Recorded home score
Away_Score	INT	Nullable	Recorded away score
Winner	VARCHAR(50)	Nullable	Winner name stored as text
Remarks	VARCHAR(100)	Nullable	Additional remarks
Fixture_ID	INT	Nullable, FK → FIXTURE	Fixture identifier

8.14 EQUIPMENT

Stores equipment names, categories and inventory quantities.

Column	Data Type	Constraint	Description
Equipment_ID	INT	PRIMARY KEY	Equipment identifier
Equipment_Name	VARCHAR(50)	Nullable	Equipment name
Category	VARCHAR(50)	Nullable	Equipment category label
Quantity	INT	Nullable	Recorded inventory quantity

8.15 EQUIPMENT_ISSUE

Stores member equipment issues and return dates.

Column	Data Type	Constraint	Description
Issue_ID	INT	PRIMARY KEY	Equipment issue identifier
Issue_Date	DATE	Nullable	Date equipment was issued
Return_Date	DATE	Nullable	Recorded return date
Condition	VARCHAR(30)	Nullable	Equipment condition label
Member_ID	INT	Nullable, FK → MEMBER	Member identifier
Equipment_ID	INT	Nullable, FK → EQUIPMENT	Equipment identifier

9. SQL Commands Used (DDL, DML)

9.1 Database Creation

```
CREATE DATABASE sports_club;  
USE sports_club;
```

9.2 Table Creation (DDL)

The following definitions reproduce the supplied MySQL script. Primary and foreign keys enforce record identity and parent references.

1. MEMBERSHIP_PLAN

```
CREATE TABLE MEMBERSHIP_PLAN (  
    Plan_ID INT PRIMARY KEY,  
    Plan_Name VARCHAR(50),  
    Duration INT,  
    Fee_Amount DECIMAL(10,2),  
    Description VARCHAR(100)  
);
```

2. MEMBER

```
CREATE TABLE MEMBER (  
    Member_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    DOB DATE,  
    Gender VARCHAR(10),  
    Phone VARCHAR(15),  
    Email VARCHAR(50),  
    Address VARCHAR(100),  
    Join_Date DATE,  
    Status VARCHAR(20),  
    Plan_ID INT,  
    FOREIGN KEY (Plan_ID) REFERENCES MEMBERSHIP_PLAN(Plan_ID)  
);
```

3. PAYMENT

```
CREATE TABLE PAYMENT (  
    Payment_ID INT PRIMARY KEY,  
    Payment_Date DATE,  
    Amount DECIMAL(10,2),  
    Remarks VARCHAR(100),  
    Mode VARCHAR(20),  
    Member_ID INT,  
    FOREIGN KEY (Member_ID) REFERENCES MEMBER(Member_ID)  
);
```

4. SPORT

```
CREATE TABLE SPORT (  
    Sport_ID INT PRIMARY KEY,  
    Sport_Name VARCHAR(50),  
    Description VARCHAR(100)  
);
```

9.2 Table Creation (continued)

5. MEMBER_SPORT

```
CREATE TABLE MEMBER_SPORT (  
    Member_ID INT,  
    Sport_ID INT,  
    PRIMARY KEY (Member_ID, Sport_ID),  
    FOREIGN KEY (Member_ID) REFERENCES MEMBER(Member_ID),  
    FOREIGN KEY (Sport_ID) REFERENCES SPORT(Sport_ID)  
);
```

6. COACH

```
CREATE TABLE COACH (  
    Coach_ID INT PRIMARY KEY,  
    Coach_Name VARCHAR(50),  
    Phone VARCHAR(15),  
    Email VARCHAR(50),  
    Experience INT  
);
```

7. TEAM

```
CREATE TABLE TEAM (  
    Team_ID INT PRIMARY KEY,  
    Team_Name VARCHAR(50),  
    Sport_ID INT,  
    Coach_ID INT,  
    FOREIGN KEY (Sport_ID) REFERENCES SPORT(Sport_ID),  
    FOREIGN KEY (Coach_ID) REFERENCES COACH(Coach_ID)  
);
```

8. FACILITY

```
CREATE TABLE FACILITY (  
    Facility_ID INT PRIMARY KEY,  
    Facility_Name VARCHAR(50),  
    Location VARCHAR(50),  
    Type VARCHAR(30),  
    Availability VARCHAR(20)  
);
```

9.2 Table Creation (continued)

9. TRAINING_SESSION

```
CREATE TABLE TRAINING_SESSION (
    Session_ID INT PRIMARY KEY,
    Session_Date DATE,
    Start_Time TIME,
    End_Time TIME,
    Team_ID INT,
    Facility_ID INT,
    FOREIGN KEY (Team_ID) REFERENCES TEAM(Team_ID),
    FOREIGN KEY (Facility_ID) REFERENCES FACILITY(Facility_ID)
);
```

10. TOURNAMENT

```
CREATE TABLE TOURNAMENT (
    Tournament_ID INT PRIMARY KEY,
    Tournament_Name VARCHAR(50),
    Sport_ID INT,
    Start_Date DATE,
    End_Date DATE,
    Description VARCHAR(100),
    FOREIGN KEY (Sport_ID) REFERENCES SPORT(Sport_ID)
);
```

11. PARTICIPANT

```
CREATE TABLE PARTICIPANT (
    Participant_ID INT PRIMARY KEY,
    Tournament_ID INT,
    Member_ID INT,
    Team_ID INT,
    Role VARCHAR(30),
    FOREIGN KEY (Tournament_ID) REFERENCES TOURNAMENT(Tournament_ID),
    FOREIGN KEY (Member_ID) REFERENCES MEMBER(Member_ID),
    FOREIGN KEY (Team_ID) REFERENCES TEAM(Team_ID)
);
```

12. FIXTURE

```
CREATE TABLE FIXTURE (
    Fixture_ID INT PRIMARY KEY,
    Fixture_Date DATE,
    Time TIME,
    Venue VARCHAR(50),
    Tournament_ID INT,
    FOREIGN KEY (Tournament_ID) REFERENCES TOURNAMENT(Tournament_ID)
);
```

9.2 Table Creation (continued)

13. RESULT

```
CREATE TABLE RESULT (  
    Result_ID INT PRIMARY KEY,  
    Home_Score INT,  
    Away_Score INT,  
    Winner VARCHAR(50),  
    Remarks VARCHAR(100),  
    Fixture_ID INT,  
    FOREIGN KEY (Fixture_ID) REFERENCES FIXTURE(Fixture_ID)  
);
```

14. EQUIPMENT

```
CREATE TABLE EQUIPMENT (  
    Equipment_ID INT PRIMARY KEY,  
    Equipment_Name VARCHAR(50),  
    Category VARCHAR(50),  
    Quantity INT  
);
```

15. EQUIPMENT_ISSUE

```
CREATE TABLE EQUIPMENT_ISSUE (  
    Issue_ID INT PRIMARY KEY,  
    Issue_Date DATE,  
    Return_Date DATE,  
    `Condition` VARCHAR(30),  
    Member_ID INT,  
    Equipment_ID INT,  
    FOREIGN KEY (Member_ID) REFERENCES MEMBER(Member_ID),  
    FOREIGN KEY (Equipment_ID) REFERENCES EQUIPMENT(Equipment_ID)  
);
```

9.3 Data Insertion (DML)

The supplied script inserts ten records into every table. The statements below show representative records from the original sample data. Parent records must be inserted before child records that reference them.

MEMBERSHIP_PLAN

```
INSERT INTO MEMBERSHIP_PLAN VALUES
(1, 'Basic', 3, 1500, 'Basic Plan');
```

MEMBER

```
INSERT INTO MEMBER VALUES
(1, 'Rahul', '2004-05-12', 'Male', '9876543210',
'rahul@gmail.com', 'Hyderabad', '2026-01-10', 'Active', 1);
```

MEMBER_SPORT

```
INSERT INTO MEMBER_SPORT VALUES
(1, 1);
```

PAYMENT

```
INSERT INTO PAYMENT VALUES
(1, '2026-01-10', 1500, 'Membership Fee', 'Cash', 1);
```

EQUIPMENT_ISSUE

```
INSERT INTO EQUIPMENT_ISSUE VALUES
(1, '2026-09-01', '2026-09-02', 'Good', 1, 1);
```

9.4 Sample Database Summary

Measure	Value
Number of tables	15
Records in each table	10
Total sample records	150
Active / inactive members	9 / 1
Total recorded payments	INR 38,500.00
Total recorded equipment quantity	200

The outputs on the following pages are derived from the original SQL dataset, before the additional UI demonstration inserts and deletes.

9.5 Sample Outputs

MEMBERSHIP_PLAN

```
SELECT * FROM MEMBERSHIP_PLAN ORDER BY Plan_ID;
```

Plan_ID	Plan_Name	Duration	Fee_Amount	Description
1	Basic	3	1500	Basic Plan
2	Standard	6	2800	Standard Plan
3	Premium	12	5000	Premium Plan
4	Student	6	2000	Student Plan
5	Gold	12	7000	Gold Plan
6	Silver	6	3500	Silver Plan
7	Family	12	9000	Family Plan
8	Monthly	1	600	Monthly Plan
9	Quarterly	3	1600	Quarterly Plan
10	Annual	12	5500	Annual Plan

MEMBER

```
SELECT Member_ID, Name, DOB, Gender, Join_Date, Status, Plan_ID
FROM MEMBER ORDER BY Member_ID;
```

Member_ID	Name	DOB	Gender	Join_Date	Status	Plan_ID
1	Rahul	2004-05-12	Male	2026-01-10	Active	1
2	Priya	2005-02-18	Female	2026-01-15	Active	2
3	Arjun	2003-08-20	Male	2026-02-01	Active	3
4	Sneha	2004-11-10	Female	2026-02-05	Active	4
5	Kiran	2002-03-25	Male	2026-02-10	Active	5
6	Anu	2005-06-14	Female	2026-03-01	Active	6
7	Vivek	2003-09-17	Male	2026-03-05	Active	7
8	Divya	2004-12-22	Female	2026-03-10	Active	8
9	Rohan	2002-07-08	Male	2026-03-15	Inactive	9
10	Meena	2005-01-30	Female	2026-03-20	Active	10

```
SELECT Member_ID, Phone, Email, Address
FROM MEMBER ORDER BY Member_ID;
```

Member_ID	Phone	Email	Address
1	9876543210	rahul@gmail.com	Hyderabad
2	9876543211	priya@gmail.com	Secunderabad
3	9876543212	arjun@gmail.com	Kukatpally
4	9876543213	sneha@gmail.com	Madhapur
5	9876543214	kiran@gmail.com	Gachibowli
6	9876543215	anu@gmail.com	Miyapur
7	9876543216	vivek@gmail.com	Ameerpet
8	9876543217	divya@gmail.com	Begumpet
9	9876543218	rohan@gmail.com	Banjara Hills
10	9876543219	meena@gmail.com	Kondapur

9.5 Sample Outputs (continued)

PAYMENT

SELECT * FROM PAYMENT ORDER BY Payment_ID;

Payment_ID	Payment_Date	Amount	Remarks	Mode	Member_ID
1	2026-01-10	1500	Membership Fee	Cash	1
2	2026-01-15	2800	Membership Fee	UPI	2
3	2026-02-01	5000	Membership Fee	Card	3
4	2026-02-05	2000	Membership Fee	UPI	4
5	2026-02-10	7000	Membership Fee	Card	5
6	2026-03-01	3500	Membership Fee	Cash	6
7	2026-03-05	9000	Membership Fee	UPI	7
8	2026-03-10	600	Monthly Fee	Cash	8
9	2026-03-15	1600	Membership Fee	UPI	9
10	2026-03-20	5500	Membership Fee	Card	10

SPORT

SELECT * FROM SPORT ORDER BY Sport_ID;

Sport_ID	Sport_Name	Description
1	Cricket	Outdoor Sport
2	Football	Team Sport
3	Basketball	Indoor Sport
4	Tennis	Racket Sport
5	Badminton	Racket Sport
6	Volleyball	Team Sport
7	Swimming	Water Sport
8	Table Tennis	Indoor Sport
9	Athletics	Track Sport
10	Kabaddi	Team Sport

MEMBER_SPORT

SELECT * FROM MEMBER_SPORT ORDER BY Member_ID, Sport_ID;

Member_ID	Sport_ID
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10

9.5 Sample Outputs (continued)

COACH

SELECT * FROM COACH ORDER BY Coach_ID;

Coach_ID	Coach_Name	Phone	Email	Experience
1	Ramesh	9000000001	ramesh@gmail.com	8
2	Suresh	9000000002	suresh@gmail.com	6
3	Mahesh	9000000003	mahesh@gmail.com	10
4	Raj	9000000004	raj@gmail.com	7
5	Karthik	9000000005	karthik@gmail.com	5
6	Anil	9000000006	anil@gmail.com	9
7	Vijay	9000000007	vijay@gmail.com	12
8	Ajay	9000000008	ajay@gmail.com	4
9	Ravi	9000000009	ravi@gmail.com	11
10	Mohan	9000000010	mohan@gmail.com	6

TEAM

SELECT * FROM TEAM ORDER BY Team_ID;

Team_ID	Team_Name	Sport_ID	Coach_ID
1	Warriors	1	1
2	FC Stars	2	2
3	Hoopers	3	3
4	Ace Team	4	4
5	Smashers	5	5
6	Volley Stars	6	6
7	Aqua Team	7	7
8	Table Kings	8	8
9	Speed Runners	9	9
10	Kabaddi Kings	10	10

9.5 Sample Outputs (continued)

FACILITY

SELECT * FROM FACILITY ORDER BY Facility_ID;

Facility_ID	Facility_Name	Location	Type	Availability
1	Cricket Ground	Block A	Ground	Available
2	Football Ground	Block B	Ground	Available
3	Basketball Court	Block C	Court	Available
4	Tennis Court	Block D	Court	Available
5	Badminton Court	Block E	Court	Available
6	Volleyball Court	Block F	Court	Available
7	Swimming Pool	Block G	Pool	Available
8	Table Tennis Hall	Block H	Hall	Available
9	Athletics Track	Block I	Track	Available
10	Kabaddi Ground	Block J	Ground	Available

TRAINING_SESSION

SELECT * FROM TRAINING_SESSION ORDER BY Session_ID;

Session_ID	Session_Date	Start_Time	End_Time	Team_ID	Facility_ID
1	2026-09-01	09:00:00	11:00:00	1	1
2	2026-09-02	10:00:00	12:00:00	2	2
3	2026-09-03	09:00:00	11:00:00	3	3
4	2026-09-04	10:00:00	12:00:00	4	4
5	2026-09-05	09:00:00	11:00:00	5	5
6	2026-09-06	10:00:00	12:00:00	6	6
7	2026-09-07	08:00:00	10:00:00	7	7
8	2026-09-08	09:00:00	11:00:00	8	8
9	2026-09-09	07:00:00	09:00:00	9	9
10	2026-09-10	10:00:00	12:00:00	10	10

9.5 Sample Outputs (continued)

TOURNAMENT

```
SELECT Tournament_ID, Tournament_Name, Sport_ID, Start_Date, End_Date
FROM TOURNAMENT ORDER BY Tournament_ID;
```

Tournament_ID	Tournament_Name	Sport_ID	Start_Date	End_Date
1	Cricket Cup	1	2026-10-01	2026-10-05
2	Football League	2	2026-10-06	2026-10-10
3	Basketball Cup	3	2026-10-11	2026-10-15
4	Tennis Open	4	2026-10-16	2026-10-20
5	Badminton Cup	5	2026-10-21	2026-10-25
6	Volleyball League	6	2026-10-26	2026-10-30
7	Swimming Meet	7	2026-11-01	2026-11-03
8	Table Tennis Cup	8	2026-11-04	2026-11-06
9	Athletics Meet	9	2026-11-07	2026-11-09
10	Kabaddi Cup	10	2026-11-10	2026-11-15

Description values are the respective sport followed by "Tournament", as shown in the original insert statements.

PARTICIPANT

```
SELECT * FROM PARTICIPANT ORDER BY Participant_ID;
```

Participant_ID	Tournament_ID	Member_ID	Team_ID	Role
1	1	1	1	Player
2	2	2	2	Player
3	3	3	3	Player
4	4	4	4	Player
5	5	5	5	Player
6	6	6	6	Player
7	7	7	7	Player
8	8	8	8	Player
9	9	9	9	Player
10	10	10	10	Player

9.5 Sample Outputs (continued)

FIXTURE

SELECT * FROM FIXTURE ORDER BY Fixture_ID;

Fixture_ID	Fixture_Date	Time	Venue	Tournament_ID
1	2026-10-01	10:00:00	Ground 1	1
2	2026-10-06	10:00:00	Ground 2	2
3	2026-10-11	11:00:00	Court 1	3
4	2026-10-16	11:00:00	Court 2	4
5	2026-10-21	10:00:00	Court 3	5
6	2026-10-26	10:00:00	Court 4	6
7	2026-11-01	09:00:00	Pool 1	7
8	2026-11-04	10:00:00	Hall 1	8
9	2026-11-07	08:00:00	Track 1	9
10	2026-11-10	10:00:00	Ground 3	10

RESULT

SELECT * FROM RESULT ORDER BY Result_ID;

Result_ID	Home_Score	Away_Score	Winner	Remarks	Fixture_ID
1	120	110	Warriors	Good Match	1
2	2	1	FC Stars	Close Match	2
3	80	75	Hoopers	Good Match	3
4	6	4	Ace Team	Good Match	4
5	21	18	Smashers	Close Match	5
6	3	2	Volley Stars	Close Match	6
7	5	3	Aqua Team	Good Match	7
8	11	9	Table Kings	Good Match	8
9	10	8	Speed Runners	Good Match	9
10	25	20	Kabaddi Kings	Close Match	10

9.5 Sample Outputs (continued)

EQUIPMENT

SELECT * FROM EQUIPMENT ORDER BY Equipment_ID;

Equipment_ID	Equipment_Name	Category	Quantity
1	Cricket Bat	Cricket	20
2	Football	Football	15
3	Basketball	Basketball	15
4	Tennis Racket	Tennis	20
5	Badminton Racket	Badminton	25
6	Volleyball	Volleyball	15
7	Swimming Goggles	Swimming	20
8	Table Tennis Bat	Table Tennis	20
9	Running Shoes	Athletics	30
10	Kabaddi Shoes	Kabaddi	20

EQUIPMENT_ISSUE

SELECT * FROM EQUIPMENT_ISSUE ORDER BY Issue_ID;

Issue_ID	Issue_Date	Return_Date	Condition	Member_ID	Equipment_ID
1	2026-09-01	2026-09-02	Good	1	1
2	2026-09-02	2026-09-03	Good	2	2
3	2026-09-03	2026-09-04	Good	3	3
4	2026-09-04	2026-09-05	Good	4	4
5	2026-09-05	2026-09-06	Good	5	5
6	2026-09-06	2026-09-07	Good	6	6
7	2026-09-07	2026-09-08	Good	7	7
8	2026-09-08	2026-09-09	Good	8	8
9	2026-09-09	2026-09-10	Good	9	9
10	2026-09-10	2026-09-11	Good	10	10

10. Queries with Outputs

These queries use the original 150-row dataset. Outputs were reproduced locally from the supplied inserts; they are tabulated results rather than new MySQL screenshots.

Query 1: Display all active members

```
SELECT Member_ID, Name, Status
FROM MEMBER
WHERE Status = 'Active'
ORDER BY Member_ID;
```

Member_ID	Name	Status
1	Rahul	Active
2	Priya	Active
3	Arjun	Active
4	Sneha	Active
5	Kiran	Active
6	Anu	Active
7	Vivek	Active
8	Divya	Active
10	Meena	Active

Result: Nine of the ten members are active. Rohan is marked Inactive.

Query 2: Member names with membership plans

```
SELECT m.Name, p.Plan_Name, p.Duration, p.Fee_Amount
FROM MEMBER m
JOIN MEMBERSHIP_PLAN p ON m.Plan_ID = p.Plan_ID
ORDER BY m.Member_ID;
```

Name	Plan_Name	Duration	Fee_Amount
Rahul	Basic	3	1500
Priya	Standard	6	2800
Arjun	Premium	12	5000
Sneha	Student	6	2000
Kiran	Gold	12	7000
Anu	Silver	6	3500
Vivek	Family	12	9000
Divya	Monthly	1	600
Rohan	Quarterly	3	1600
Meena	Annual	12	5500

Result: The join displays each member's selected plan, duration and fee.

10. Queries with Outputs (continued)

Query 3: Member sport registrations

```
SELECT m.Name, s.Sport_Name
FROM MEMBER_SPORT ms
JOIN MEMBER m ON ms.Member_ID = m.Member_ID
JOIN SPORT s ON ms.Sport_ID = s.Sport_ID
ORDER BY m.Member_ID;
```

Name	Sport_Name
Rahul	Cricket
Priya	Football
Arjun	Basketball
Sneha	Tennis
Kiran	Badminton
Anu	Volleyball
Vivek	Swimming
Divya	Table Tennis
Rohan	Athletics
Meena	Kabaddi

Result: The junction table connects members to registered sports.

Query 4: Total number of members

```
SELECT COUNT(*) AS Total_Members FROM MEMBER;
```

Total_Members
10

Result: The original sample database contains ten members.

Query 5: Total amount collected

```
SELECT SUM(Amount) AS Total_Collection FROM PAYMENT;
```

Total_Collection
38500

Result: The ten supplied payment records total INR 38,500.00.

Query 6: Highest and average payment

```
SELECT MAX(Amount) AS Highest_Payment,
       AVG(Amount) AS Average_Payment
FROM PAYMENT;
```

Highest_Payment	Average_Payment
9000	3850.0

Result: The highest payment is INR 9,000.00 and the average is INR 3,850.00.

10. Queries with Outputs (continued)

Query 7: Payments grouped by mode

```
SELECT Mode, COUNT(*) AS Payment_Count,
       SUM(Amount) AS Total_Amount
FROM PAYMENT
GROUP BY Mode ORDER BY Mode;
```

Mode	Payment_Count	Total_Amount
Card	3	17500
Cash	3	5600
UPI	4	15400

Result: Card: 3 payments / INR 17,500; Cash: 3 / INR 5,600; UPI: 4 / INR 15,400.

Query 8: Teams and their coaches

```
SELECT t.Team_Name, s.Sport_Name, c.Coach_Name
FROM TEAM t
JOIN SPORT s ON t.Sport_ID = s.Sport_ID
JOIN COACH c ON t.Coach_ID = c.Coach_ID
ORDER BY t.Team_ID;
```

Team_Name	Sport_Name	Coach_Name
Warriors	Cricket	Ramesh
FC Stars	Football	Suresh
Hoopers	Basketball	Mahesh
Ace Team	Tennis	Raj
Smashers	Badminton	Karthik
Volley Stars	Volleyball	Anil
Aqua Team	Swimming	Vijay
Table Kings	Table Tennis	Ajay
Speed Runners	Athletics	Ravi
Kabaddi Kings	Kabaddi	Mohan

Result: Each sample team is displayed with its sport and assigned coach.

10. Queries with Outputs (continued)

Query 9: Training sessions with facilities

```
SELECT t.Team_Name, ts.Session_Date,
       ts.Start_Time, ts.End_Time, f.Facility_Name
FROM TRAINING_SESSION ts
JOIN TEAM t ON ts.Team_ID = t.Team_ID
JOIN FACILITY f ON ts.Facility_ID = f.Facility_ID
ORDER BY ts.Session_ID;
```

Team_Name	Session_Date	Start_Time	End_Time	Facility_Name
Warriors	2026-09-01	09:00:00	11:00:00	Cricket Ground
FC Stars	2026-09-02	10:00:00	12:00:00	Football Ground
Hoopers	2026-09-03	09:00:00	11:00:00	Basketball Court
Ace Team	2026-09-04	10:00:00	12:00:00	Tennis Court
Smashers	2026-09-05	09:00:00	11:00:00	Badminton Court
Volley Stars	2026-09-06	10:00:00	12:00:00	Volleyball Court
Aqua Team	2026-09-07	08:00:00	10:00:00	Swimming Pool
Table Kings	2026-09-08	09:00:00	11:00:00	Table Tennis Hall
Speed Runners	2026-09-09	07:00:00	09:00:00	Athletics Track
Kabaddi Kings	2026-09-10	10:00:00	12:00:00	Kabaddi Ground

Result: The result joins training times to the team and booked facility.

Query 10: Tournament winners

```
SELECT tr.Tournament_Name, fx.Fixture_Date, r.Winner
FROM TOURNAMENT tr
JOIN FIXTURE fx ON tr.Tournament_ID = fx.Tournament_ID
JOIN RESULT r ON fx.Fixture_ID = r.Fixture_ID
ORDER BY tr.Tournament_ID;
```

Tournament_Name	Fixture_Date	Winner
Cricket Cup	2026-10-01	Warriors
Football League	2026-10-06	FC Stars
Basketball Cup	2026-10-11	Hoopers
Tennis Open	2026-10-16	Ace Team
Badminton Cup	2026-10-21	Smashers
Volleyball League	2026-10-26	Volley Stars
Swimming Meet	2026-11-01	Aqua Team
Table Tennis Cup	2026-11-04	Table Kings
Athletics Meet	2026-11-07	Speed Runners
Kabaddi Cup	2026-11-10	Kabaddi Kings

Result: Each stored result is traced back to its fixture and tournament.

10. Queries with Outputs (continued)

Query 11: Equipment issued to members

```
SELECT m.Name, e.Equipment_Name, ei.Issue_Date, ei.Return_Date
FROM EQUIPMENT_ISSUE ei
JOIN MEMBER m ON ei.Member_ID = m.Member_ID
JOIN EQUIPMENT e ON ei.Equipment_ID = e.Equipment_ID
ORDER BY ei.Issue_ID;
```

Name	Equipment_Name	Issue_Date	Return_Date
Rahul	Cricket Bat	2026-09-01	2026-09-02
Priya	Football	2026-09-02	2026-09-03
Arjun	Basketball	2026-09-03	2026-09-04
Sneha	Tennis Racket	2026-09-04	2026-09-05
Kiran	Badminton Racket	2026-09-05	2026-09-06
Anu	Volleyball	2026-09-06	2026-09-07
Vivek	Swimming Goggles	2026-09-07	2026-09-08
Divya	Table Tennis Bat	2026-09-08	2026-09-09
Rohan	Running Shoes	2026-09-09	2026-09-10
Meena	Kabaddi Shoes	2026-09-10	2026-09-11

Result: The query lists members, issued equipment and recorded return dates.

Query 12: Total recorded equipment quantity

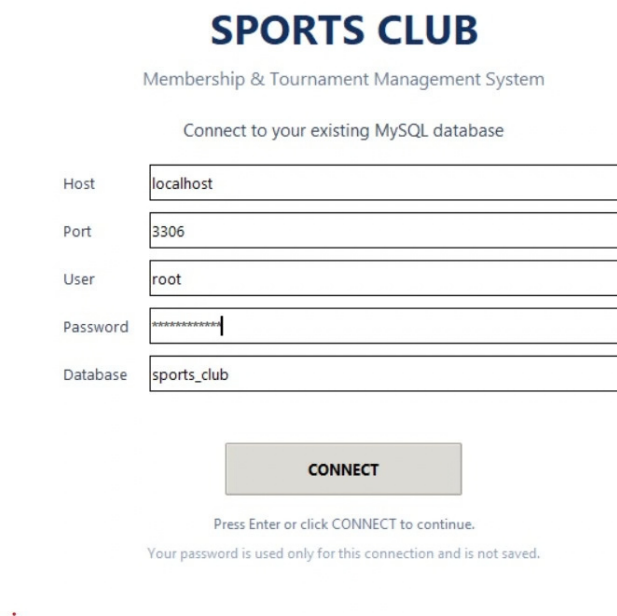
```
SELECT SUM(Quantity) AS Total_Equipment_Quantity
FROM EQUIPMENT;
```

Total_Equipment_Quantity
200

Result: The EQUIPMENT table records a total quantity of 200 units. This is a stored inventory total, not a calculated available balance.

11. UI Design and Screenshots

The desktop interface is developed using Python Tkinter and connects to the sports_club MySQL database. Presentation 3 shows the connection screen and the view, insert and delete workflows. The screenshots below are extracted from the supplied presentation.



SPORTS CLUB
Membership & Tournament Management System

Connect to your existing MySQL database

Host	localhost
Port	3306
User	root
Password	*****
Database	sports_club

CONNECT

Press Enter or click CONNECT to continue.
Your password is used only for this connection and is not saved.

Figure 11.1: Sports Club database connection screen

The user enters host, port, username, password and database name, then clicks CONNECT. The displayed connection values are localhost, 3306, root and sports_club. The screen states that the password is used for the current connection and is not saved.

11.1 Viewing and Inserting Member Records

Members
View, insert and delete records from MEMBER

Refresh Add Record Delete Selected Double-click a row to view details

Member ID	Name	DOB	Gender	Phone	Email	Address	Join Date	Status	
1	Rahul	2004-05-12	Male	9876543210	rahul@gmail.com	Hyderabad	2026-01-10	Active	1
2	Priya	2003-02-18	Female	9876543211	priya@gmail.com	Secunderabad	2026-01-15	Active	2
3	Arjun	2003-08-20	Male	9876543212	arjun@gmail.com	Kukatpally	2026-02-01	Active	3
4	Sneha	2004-11-10	Female	9876543213	sneha@gmail.com	Madhapur	2026-02-05	Active	4
5	Kiran	2002-03-25	Male	9876543214	kiran@gmail.com	Gachibowli	2026-02-10	Active	5
6	Anu	2005-06-14	Female	9876543215	anu@gmail.com	Miyapur	2026-03-01	Active	6
7	Vivek	2003-09-17	Male	9876543216	vivek@gmail.com	Ameerpet	2026-03-05	Active	7
8	Divya	2004-12-22	Female	9876543217	divya@gmail.com	Begumpet	2026-03-10	Active	8
9	Rohan	2002-07-08	Male	9876543218	rohan@gmail.com	Banjara Hills	2026-03-15	Inactive	9
10	Meena	2005-01-30	Female	9876543219	meena@gmail.com	Kondapur	2026-03-20	Active	10

Figure 11.2: MEMBER table with the ten original sample members

The Members screen displays the selected table in a grid. Refresh reloads the records, Add Record opens the insertion workflow, and Delete Selected operates on the selected record.

Members
View, insert and delete records from MEMBER

Refresh Add Record Delete Selected Double-click a row to view details

Member ID	Name	DOB	Gender	Phone	Email	Address	Join Date	Status	
1	Rahul	2004-05-12	Male	9876543210	rahul@gmail.com	Hyderabad	2026-01-10	Active	1
2	Priya	2003-02-18	Female	9876543211	priya@gmail.com	Secunderabad	2026-01-15	Active	2
3	Arjun	2003-08-20	Male	9876543212	arjun@gmail.com	Kukatpally	2026-02-01	Active	3
4	Sneha	2004-11-10	Female	9876543213	sneha@gmail.com	Madhapur	2026-02-05	Active	4
5	Kiran	2002-03-25	Male	9876543214	kiran@gmail.com	Gachibowli	2026-02-10	Active	5
6	Anu	2005-06-14	Female	9876543215	anu@gmail.com	Miyapur	2026-03-01	Active	6
7	Vivek	2003-09-17	Male	9876543216	vivek@gmail.com	Ameerpet	2026-03-05	Active	7
8	Divya	2004-12-22	Female	9876543217	divya@gmail.com	Begumpet	2026-03-10	Active	8
9	Rohan	2002-07-08	Male	9876543218	rohan@gmail.com	Banjara Hills	2026-03-15	Inactive	9
10	Meena	2005-01-30	Female	9876543219	meena@gmail.com	Kondapur	2026-03-20	Active	10

Figure 11.3: Member records before insertion

Members
View, insert and delete records from MEMBER

Refresh Add Record Delete Selected Double-click a row to view details

Member ID	Name	DOB	Gender	Phone	Email	Address	Join Date	Status	
1	Rahul	2004-05-12	Male	9876543210	rahul@gmail.com	Hyderabad	2026-01-10	Active	1
2	Priya	2003-02-18	Female	9876543211	priya@gmail.com	Secunderabad	2026-01-15	Active	2
3	Arjun	2003-08-20	Male	9876543212	arjun@gmail.com	Kukatpally	2026-02-01	Active	3
4	Sneha	2004-11-10	Female	9876543213	sneha@gmail.com	Madhapur	2026-02-05	Active	4
5	Kiran	2002-03-25	Male	9876543214	kiran@gmail.com	Gachibowli	2026-02-10	Active	5
6	Anu	2005-06-14	Female	9876543215	anu@gmail.com	Miyapur	2026-03-01	Active	6
7	Vivek	2003-09-17	Male	9876543216	vivek@gmail.com	Ameerpet	2026-03-05	Active	7
8	Divya	2004-12-22	Female	9876543217	divya@gmail.com	Begumpet	2026-03-10	Active	8
9	Rohan	2002-07-08	Male	9876543218	rohan@gmail.com	Banjara Hills	2026-03-15	Inactive	9
10	Meena	2005-01-30	Female	9876543219	meena@gmail.com	Kondapur	2026-03-20	Active	10
11	Reethu	2006-10-04	Female	9390153889	mreethu@gmail.com	Hyderabad	2026-10-06	Active	4
12	tanishqa	2007-02-07	Female	7896876765	tan@gmail.com	Secunderabad	2026-10-06	Active	5
13	Megha	2007-01-01	Female	7689657865	megha@gmail.com	warangal	2026-10-06	Active	9

Figure 11.4: Member records after additional demonstration inserts

The additional rows in Figure 11.4 belong to the UI demonstration. They are not included in the original ten-row MEMBER dataset used for the SQL outputs in this report.

11.2 Deleting a Payment Record

The deletion images in Presentation 3 show the PAYMENT table. Although the slide text refers to a customer and a confirmation, the visible dialog reports successful deletion. The captions here describe the actual screenshot content.

View, insert and delete records from PAYMENT

Refresh Add Record Delete Selected Double-click a row to view details

Payment ID	Payment Date	Amount	Remarks	Mode	Member ID
1	2026-01-10	1500.00	Membership Fee	Cash	1
2	2026-01-15	2800.00	Membership Fee	UPI	2
3	2026-02-01	5000.00	Membership Fee	Card	3
4	2026-02-05	2000.00	Membership Fee	UPI	4
5	2026-02-10	7000.00	Membership Fee	Card	5
6	2026-03-01		Membership Fee	Cash	6
7	2026-03-05		Membership Fee	UPI	7
8	2026-03-10		Monthly Fee	Cash	8
9	2026-03-15		Membership Fee	UPI	9
10	2026-03-20		Membership Fee	Card	10
11	2026-10-06	7000.00	Monthly Fee	Cash	13
12	2026-10-06	4000.00	Membership Fee	Cash	11

Deleted Record deleted successfully. OK

Figure 11.5: PAYMENT table with the “Record deleted successfully” message

View, insert and delete records from PAYMENT

Refresh Add Record Delete Selected Double-click a row to view details

Payment ID	Payment Date	Amount	Remarks	Mode	Member ID
1	2026-01-10	1500.00	Membership Fee	Cash	1
2	2026-01-15	2800.00	Membership Fee	UPI	2
3	2026-02-01	5000.00	Membership Fee	Card	3
4	2026-02-05	2000.00	Membership Fee	UPI	4
5	2026-02-10	7000.00	Membership Fee	Card	5
6	2026-03-01	3500.00	Membership Fee	Cash	6
7	2026-03-05	9000.00	Membership Fee	UPI	7
8	2026-03-10	600.00	Monthly Fee	Cash	8
9	2026-03-15	1600.00	Membership Fee	UPI	9
10	2026-03-20	5500.00	Membership Fee	Card	10
12	2026-10-06	4000.00	Membership Fee	Cash	11

Figure 11.6: PAYMENT table after the demonstrated deletion

11.3 Interface Workflow

- Connect to the existing sports_club database using valid MySQL credentials.
- Choose a table and view its records in the grid.
- Use Add Record to enter and save a new record.
- Select a record and use Delete Selected to remove it.
- Use Refresh to show the current database contents.

Foreign-key restrictions apply to deletion. A referenced parent member cannot be removed until dependent records are handled. This behavior comes from the database relationships rather than the grid display.

12. Implementation Details

12.1 Technology Stack

Layer	Technology
Database	MySQL
Query Language	SQL — DDL, DML and SELECT queries
Application Language	Python 3.x
User Interface	Tkinter desktop GUI
Connector	mysql-connector-python
Client	MySQL Workbench / MySQL command-line client
Design Evidence	ER diagram and three supplied presentations

12.2 Database Setup and Connectivity

The SQL script first creates the sports_club database and selects it using USE sports_club. Parent tables are created before their dependent tables, and the sample inserts follow the same dependency order. The script is intended for initial setup; running CREATE DATABASE or CREATE TABLE again without removing existing objects will produce an existing-object error.

The application's connection screen requests host, port, username, password and database name. After a successful connection, the interface retrieves records from the selected table. Insert and delete actions change persistent database records, while Refresh reloads the grid.

12.3 Application Features

- Browse the selected table using a row-and-column grid.
- Insert records through the Add Record workflow.
- Delete the selected record and display the operation result.
- Reload the grid using Refresh.
- Use database primary and foreign keys to preserve identity and valid references.

The supplied attachments include SQL and interface screenshots, but not the Python source. Therefore internal implementation details such as cursor handling, form-generation logic and transaction code are not claimed as verified source-code behavior.

12.4 Key SQL Code Snippets

(a) Composite key — member sport registrations

```
CREATE TABLE MEMBER_SPORT (
    Member_ID INT,
    Sport_ID INT,
    PRIMARY KEY (Member_ID, Sport_ID),
    FOREIGN KEY (Member_ID) REFERENCES MEMBER(Member_ID),
    FOREIGN KEY (Sport_ID) REFERENCES SPORT(Sport_ID)
);
```

The composite key prevents a duplicate Member_ID–Sport_ID pair, while both foreign keys require referenced parent records when the identifiers are non-NULL.

(b) A table with two parent references — training sessions

```
CREATE TABLE TRAINING_SESSION (
    Session_ID INT PRIMARY KEY,
    Session_Date DATE,
    Start_Time TIME,
    End_Time TIME,
    Team_ID INT,
    Facility_ID INT,
    FOREIGN KEY (Team_ID) REFERENCES TEAM(Team_ID),
    FOREIGN KEY (Facility_ID) REFERENCES FACILITY(Facility_ID)
);
```

Team_ID and Facility_ID link a training session to its team and facility. The supplied table definition does not check End_Time against Start_Time or prevent overlapping sessions.

(c) Aggregation with grouping — payment summary

```
SELECT Mode, COUNT(*) AS Payment_Count,
       SUM(Amount) AS Total_Amount
FROM PAYMENT
GROUP BY Mode ORDER BY Mode;
```

COUNT summarizes the number of payments and SUM totals their amounts for each payment mode.

(d) Reserved column-name handling — equipment condition

```
SELECT Issue_ID, `Condition`, Member_ID, Equipment_ID
FROM EQUIPMENT_ISSUE
ORDER BY Issue_ID;
```

The backticks preserve the exact Condition column name used in the MySQL definition.

13. Testing

The supplied SQL was loaded into a local SQLite verification database after omitting only the MySQL database-selection statements and enabling foreign-key checks. This verified the sample rows, relational references and query results. It does not establish that the desktop application or MySQL server was newly executed for this report. The UI evidence is supplied in Presentation 3.

ID	Test Case	Expected Result	Local Result	Status
T1	Duplicate member ID	Rejected by primary key	Rejected	Pass
T2	Duplicate member-sport pair	Rejected by composite key	Rejected	Pass
T3	Unknown membership plan	Rejected by foreign key	Rejected	Pass
T4	Unknown member in PAYMENT	Rejected by foreign key	Rejected	Pass
T5	Delete referenced member	Rejected by foreign key	Rejected	Pass
T6	Total members	10	10	Pass
T7	Rows across all tables	150	150	Pass
T8	Total collection	38500	38500	Pass
T9	Active members	9	9	Pass
T10	Equipment quantity	200	200	Pass

All ten local verification checks passed. The twelve report queries were also executed against the same original sample data to produce the outputs shown in Section 10.

13.1 Validation Gaps in the Supplied DDL

- Negative amounts and quantities are not blocked by CHECK constraints.
- Phone and email duplicates are allowed because no UNIQUE constraints are defined.
- Foreign-key fields allow NULL; mandatory relationships need NOT NULL.
- Invalid date ranges, schedule overlaps and repeated result rows per fixture need further validation.

MySQL-specific NULL behavior for primary keys and exact server error messages require confirmation on the target MySQL installation. Local test results are not presented as MySQL error outputs.

14. Conclusion and Future Enhancements

14.1 Conclusion

The Sports Club Membership and Tournament Management System organizes club data into fifteen connected relational tables. The final SQL includes membership plans, members, payments, sports, member–sport registrations, coaches, teams, facilities, training sessions, tournaments, participants, fixtures, results, equipment and equipment issues.

The 150-record sample dataset demonstrates how these entities work together. Queries retrieve member plans and sports, team coaches, training facilities, tournament winners, equipment issues and payment summaries. The Python Tkinter interface shown in the supplied presentation supports connecting to MySQL, viewing records, inserting members and deleting payments.

The design is intended to satisfy 3NF under the stated dependencies. Primary and foreign keys enforce identity and references, while further business validation would strengthen the system. The project forms a useful foundation for a more complete club management application.

14.2 Future Enhancements

- Add NOT NULL, UNIQUE and CHECK constraints for required fields, phone/e-mail rules, positive payments, non-negative quantities and valid date ranges.
- Add login accounts and permissions for administrators, coaches, members, tournament managers and accounts staff.
- Track membership start/end dates, renewals, outstanding fees and reminders.
- Prevent conflicting facility bookings and invalid training time ranges.
- Create a MEMBER_TEAM table to track team membership outside tournaments.
- Store the home and away team references for fixtures and enforce one result per fixture if required.
- Add issued quantities, return status and automatic available-stock calculation for equipment.
- Add record editing, search, printable reports, backups and online payment integration.

15. References

- [1] Sports Club Membership and Tournament Management System, Presentation 1 — problem statement, scope, objectives, users and requirements. Supplied project material.
- [2] Sports Club Membership and Tournament Management System, Presentation 2 — ER diagram and initial relational schema. Supplied project material.
- [3] Sports Club Membership and Tournament Management System, Presentation 3 — database connection and view, insert and delete screenshots. Supplied project material.
- [4] Pasted text(3).txt — final sports_club MySQL table definitions and sample insert statements. Supplied project source.
- [5] Vehicle Service Centre Management System — reference report supplied for document structure and formatting.

Suggested Technical Documentation

Oracle Corporation, MySQL Reference Manual: <https://dev.mysql.com/doc/>

Python Software Foundation, Tkinter documentation:
<https://docs.python.org/3/library/tkinter.html>

Oracle Corporation, MySQL Connector/Python documentation:
<https://dev.mysql.com/doc/connector-python/en/>

The project attachments are the evidence used for this report. The technical documentation links are provided for further reading.

16. Appendix: GitHub Repository Link

The final submission repository should contain the database script, application source, ER diagram, presentations, report and relevant screenshots.

16.1 Project Repository

GitHub Repository Link: _____

A repository URL was not supplied with the project materials. Enter the actual project URL here before submission.

16.2 Recommended Repository Contents

- sports_club.sql — database creation, table definitions and sample inserts.
- Python Tkinter application file(s) and requirements.txt.
- ER diagram and relational schema.
- Presentations 1, 2 and 3.
- Project report and interface screenshots.
- README with database setup and application launch instructions.

16.3 Sample Data Overview

Table	Sample Rows
MEMBERSHIP_PLAN	10
MEMBER	10
PAYMENT	10
SPORT	10
MEMBER_SPORT	10
COACH	10
TEAM	10
FACILITY	10
TRAINING_SESSION	10
TOURNAMENT	10
PARTICIPANT	10
FIXTURE	10
RESULT	10
EQUIPMENT	10
EQUIPMENT_ISSUE	10
TOTAL	150